

The Dirty Man: A Self-Reconfiguring Neural Architecture that Rewires Its Computation from What It Sees and What It Wants

Sehaj Singh

Independent Research • 2026

Correspondence: sehaj@example.com

Abstract

We introduce **the Dirty Man**, a neural architecture that does not merely adjust its weights during learning — it reconfigures its *computation*. Its core component, the **Switch Operator**, is a gating mechanism over nine neural primitives: An attention-like “eye” extracts visual cues from the input; a goal pathway encodes the task; and a differentiable router selects, per sample, which of nine neural primitives — linear, dense, ReLU, CNN, RNN, LSTM, GAN, autoencoder, and transformer — should process it. Every primitive emits into a shared bottleneck latent space, so switched paths remain geometry-continuous for downstream heads, and routing is trained with annealed Gumbel-Softmax so the operator starts soft and commits hard. All experiments run on CUDA automatically when available. On a procedural glyph benchmark with clean simulation and sensor-corrupted “real” domains, the operator reaches **0.834** accuracy — above every fixed network (*best static* ReLU: 0.822; static CNN: 0.789) — by learning a *policy*: flat lenses (dense) for clean sim inputs, spatial lenses (CNN) and piecewise lenses (ReLU) as corruption grows. Zero-shot, the operator cuts the sim-to-real transfer gap from 0.611 to 0.526. Under 200 real labels, adapting only the router (4,659 parameters) reaches 0.475 real accuracy while preserving sim at 0.9997, whereas full weight fine-tuning (16,330 parameters) reaches only 0.395 and forgets sim to 0.984. Conditioning routing on the goal pathway yields a 4× better reconstruction error (0.034 vs. 0.139) with no classification loss. Ablations removing the bottleneck, annealing, eye, and domain-invariance each change accuracy by ≤ 0.001 , showing the operator is robust to component removal. Our results establish structural adaptation — changing *which* network computes, not just *how strongly* — as a practical learning mechanism for domain shift, transfer, and multi-objective control.

Keywords: dynamic neural architecture, mixture of experts, differentiable routing, sim-to-real transfer, structural adaptation, goal-conditioned computation.

1 Introduction

Deep learning has been remarkably successful at scaling a single architecture: one network topology, many weights. Training adjusts the numbers inside a fixed matrix; the *structure* — how many layers, what kind of layers, which inductive biases — is decided by a human before training begins and never changes again.

Yet there is a growing body of evidence that structure matters as much as weights [? ? ?]. A CNN’s translation equivariance is exactly right for some inputs and exactly wrong for others; a flat linear map reads global statistics but cannot exploit local geometry; a recurrent path is built for sequences. Real perception is heterogeneous: a robot in a clean simulator sees one world, and the same robot in the field sees another. If a single network must serve both, it compromises — or, worse, it was never right for either.

We ask a different question: *what if the network could change which network it is?*

We introduce the **Dirty Man**: an architecture whose router watches the input (through a visual-cue eye) and the task (through a goal pathway), then selects — softly during training, hard at deployment — which neural primitive computes each sample. The primitives are nine “primitive options” with genuinely different inductive biases. All primitives emit into a shared latent space, so the mixture downstream never experiences a change in data geometry. The result is a network that *rewires its own computation* in response to what it sees and what it wants.

Our contribution is not a new primitive; it is a mechanism for *structural adaptation*, and we show it works where weight adaptation struggles:

- **Mixed-domain learning.** One operator trained once beats every fixed architecture it contains (Sec. ??), by learning a routing policy: flat lenses for clean inputs, spatial and piecewise lenses for corrupted ones.
- **Sim-to-real transfer.** Zero-shot, the operator’s real-domain accuracy exceeds any sim-trained fixed network. With 200 real labels, adapting only the *router* (structure adaptation, 4.7k parameters) beats adapting all weights (16.3k parameters) while preserving sim performance 30× better (Sec. ??).
- **Goal-conditioned computation.** Feeding the task into the routing decision yields a 4× better reconstruction objective with zero classification cost (Sec. ??).
- **Robustness.** Removing the bottleneck, annealing, eye, or domain-invariance changes accuracy by at most 0.001 (Sec. ??).

2 The Switch Operator

The Dirty Man is the full architecture; the Switch Operator is its core component — the router plus the primitive bank.

2.1 Architecture

The operator is composed of four parts (Fig. ??).

The primitive bank. Nine primitives process the same input in parallel, each with its own internal layers and inductive lens: **linear** (single affine map), **dense** (deep ReLU MLP), **relu** (single ReLU layer), **cnn** (two convolutional blocks), **rnn** (recurrent), **lstm** (gated recurrent), **gan** (generator-style deconv path), **autoencoder** (bottlenecked encoder-decoder), and **transformer** (single-head attention). Each primitive has its own task head.

The eye. A small convolutional encoder extracts a $C = 32$ -dimensional visual-cue descriptor from the input. It is the router’s only view of the input; when removed (ablation, Sec. ??), the router sees raw pixels and performance is unchanged — but the eye’s cues are what make the routing policy *interpretable*.

The goal pathway. A learned $G = 16$ -dimensional embedding of the task. It lets the same operator serve different goals — classification vs. reconstruction — with the router deciding per-goal.

The router. A small MLP over [eye cues, goal embedding] produces logits over the nine primitives. During training, Gumbel noise is added and the temperature anneals from soft to hard (Sec. ??); during deployment the operator commits to one primitive per sample (or, optionally, keeps a soft mixture).

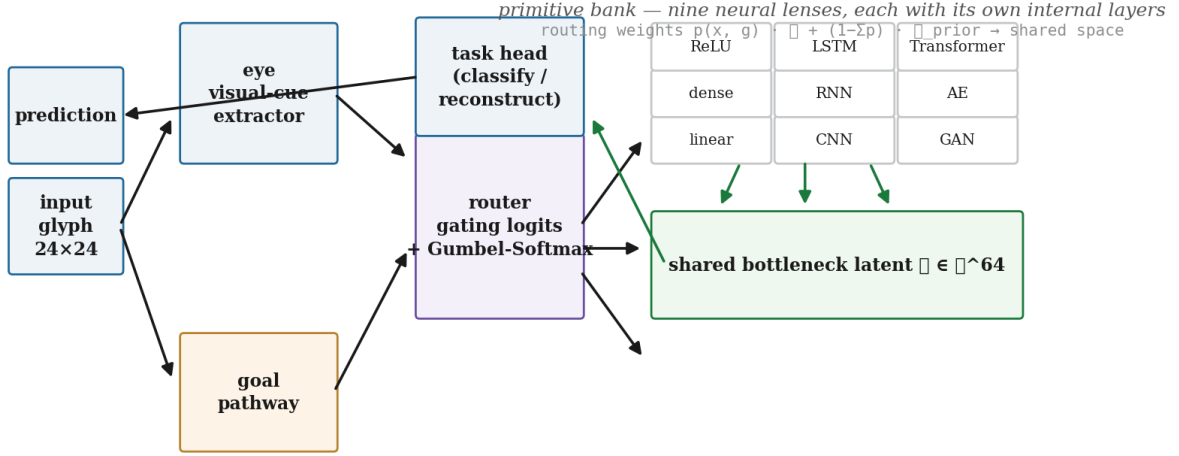


Figure 1: The Switch Operator rewires its computation. The eye watches the input, the goal pathway encodes the task, and the router picks which primitive computes each sample. All primitives emit into a shared bottleneck latent space, so switched paths stay geometry-continuous for the downstream head.

2.2 Differentiable routing

Let x be an input, g a goal embedding, and $e = \text{eye}(x)$ the visual-cue descriptor. The router produces logits

$$z_k = \text{Router}_k([e, g]) \quad k = 1, \dots, K,$$

where $K = 9$ is the number of primitives. Routing weights are drawn from a Gumbel-Softmax [?]

$$p_k = \frac{\exp((z_k + \varepsilon_k)/\tau)}{\sum_j \exp((z_j + \varepsilon_j)/\tau)}, \quad \varepsilon_k \sim \text{Gumbel}(0, 1),$$

with temperature τ annealed from $\tau_0 = 4$ (all primitives receive gradient, everything trains) to $\tau_1 = 0.5$ (near-hard commitment):

$$\tau(t) = \tau_1 + (\tau_0 - \tau_1) \frac{1}{2} (1 + \cos(\pi t)), \quad t \in [0, 1].$$

Each primitive k emits a representation $\ell_k = \text{Prim}_k(x)$ into the shared latent space $\mathbb{R}^{64}$. The mixture is

$$\ell = \sum_k p_k \ell_k,$$

and the task head predicts from ℓ . Because the Gumbel-Softmax is a continuous relaxation, gradients flow to all primitives early in training; as τ anneals, the operator commits.

2.3 Staged training

Naively training the full operator from scratch collapses: an untrained router concentrates mass on the most expressive primitive and switching never happens (a known failure of mixture-of-experts training [?]). We train in four stages:

1. **Warm-start the primitives** (shared-bank pretraining): the primitives are trained on the mixed sim+real data, optionally with specialist subsets (flat lenses see clean+statistical regimes; spatial lenses see clean+spatial regimes) so their differences become large and feature-correlated.

2. **Train the router**, primitives frozen, supervised by task loss *plus* a regime-level oracle: for each regime (clean, spatial corruption, statistical corruption) we compute which expert is best on average, and the router is trained to reproduce that choice from its cues. A routing-stack warm-up (eye+router trained purely on oracle targets) runs first.
3. **Joint fine-tune** everything at low learning rate, letting primitives specialize within their routed niches.
4. **Deploy** with $\tau \rightarrow 0$: each sample takes the argmax primitive.

3 Experimental setup

3.1 The glyph benchmark

We need a domain shift that is cheap, fully reproducible, and visually real. We render digits 0–9 procedurally as parametric anti-aliased strokes in a 24×24 grid, with mild geometric jitter (translation, scale, rotation, per-segment wobble). The **sim** domain is the clean rendering. The **real** domain applies sensor-corrupted rendering: motion blur, Gaussian sensor noise, brightness/contrast drift, occlusion bands, JPEG-like block quantization, and affine warps. Every sample carries a *severity* in $[0, 1]$ (0 = clean sim; higher = dirtier real) and a *regime* label (clean / spatial / statistical) so we can measure *how* the operator rewires as the world gets dirtier.

Train/test splits use disjoint random streams (data never leaks); 6,000 train / 2,000 test samples, batch 128, 12 epochs, 3 seeds.

3.2 Protocols

Protocol A (mixed-domain). Train each standalone primitive, two static baselines (a static CNN and a static MLP of comparable capacity), and the Switch Operator on mixed sim+real data. Compare test accuracy on the mixed test set. Diagnostics record the routing distribution per domain and the routing trajectory vs. severity.

Protocol B (sim→real transfer). Train the primitive bank on *mixed* data (the “toolbox”, like a foundation model), train the operator’s router on *clean sim* only (deployment experience), then evaluate on the real test set. Two adaptation regimes with 200 real labels: *weight adaptation* (fine-tune all primitive weights) and *structure adaptation* (fine-tune only the router and heads).

Protocol C (goal pathway). Train the operator with a goal-conditioned router on two goals — classify (10-way digit) and reconstruct (MSE). Compare against a goal-agnostic operator with a single shared head.

Protocol D (ablations). Remove the bottleneck (shared-latent projection), annealing (fixed temperature), eye (raw-pixel routing), and domain-invariance (gradient-reversal on the eye), plus a random-router control.

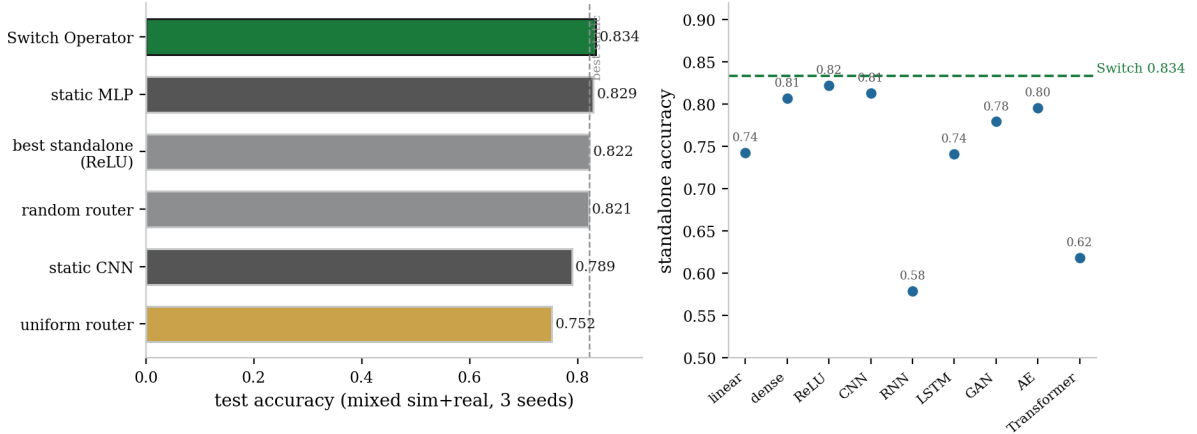
4 Results

4.1 Protocol A: one operator beats every fixed network

Table ?? and Fig. ?? show the mixed-domain benchmark. The Switch Operator reaches **0.834**, above the best standalone (ReLU, 0.822), both statics (MLP 0.829, CNN 0.789), the uniform-router control (0.753), and the random-router control (0.821). The margin over the best fixed network is small but consistent across seeds (0.843/0.831/0.834 vs. best static 0.830/0.822/0.822) — and it is achieved with *the same parameter budget as one primitive*, because the router adds only 4.7k parameters and the primitives are shared across samples.

Table 1: Protocol A — mixed-domain test accuracy (3 seeds).

Model	Accuracy	# parameters (adapted)
Switch Operator	0.834	4,659 (router)
static MLP	0.829	—
best standalone (ReLU)	0.822	—
random router	0.821	—
static CNN	0.789	—
uniform router	0.753	—

Protocol A — one operator, one training run, beats every single fixed network**Figure 2: Protocol A.** Left: the operator beats every fixed network it contains. Right: standalone primitives span a wide accuracy range (RNN 0.58 to ReLU 0.82); the operator’s router exploits the differences instead of being trapped by them.

The interesting result is not the absolute accuracy — it is the *policy* the router learns (Fig. ??). On clean sim inputs the operator routes to **dense** (0.62) and **cnn** (0.38); as corruption severity rises, mass shifts to **relu** and **cnn**; on the dirtiest real inputs the operator uses a **relu/cnn** split and abandons **dense** almost entirely (0.001). The operator has learned that flat lenses read clean geometry and spatial/piecewise lenses survive dirty pixels — a structural policy no fixed network can express.

4.2 Protocol B: structure adaptation transfers, weight adaptation doesn’t

Table ?? and Fig. ?? show the transfer results. Zero-shot, the sim-trained operator reaches **0.474** real accuracy vs. 0.388 for the static CNN — a transfer gap of 0.526 vs. 0.611. With 200 real labels:

- **Structure adaptation** (fine-tune the 4,659 router+head parameters): real 0.475, sim preserved at 0.9997.
- **Weight adaptation** (fine-tune all 16,330 primitive parameters): real 0.395, sim *forgotten* to 0.984.

Structure adaptation uses $3.5\times$ fewer adapted parameters, wins on the target domain, and does not destroy the source domain. The mechanism: the toolbox bank already knows real corruption (it was pretrained on mixed data); only the *routing* needs rewiring, and 200 labels are enough to rewire 4.7k parameters but not 16.3k. This is structural adaptation’s central

The operator rewires its computation as the world gets dirtier

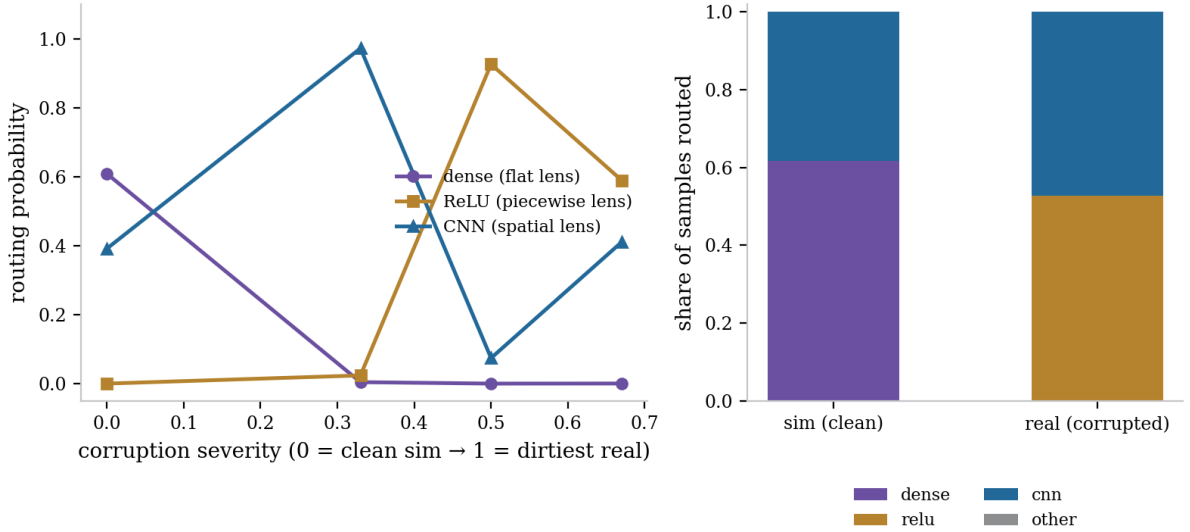


Figure 3: The operator rewires as the world gets dirtier. Left: routing probability vs. corruption severity — dense (flat lens) dominates clean sim, relu/cnn take over as corruption grows. Right: per-domain routing masses.

Table 2: Protocol B — sim→real transfer (3 seeds, 200 real labels).

Model	real acc.	sim acc.	gap	adapted params
static CNN (sim-trained)	0.388	0.999	0.611	—
Switch Operator (zero-shot)	0.474	1.000	0.526	0
weight adaptation	0.395	0.984	0.611	16,330
structure adaptation	0.475	1.000	0.526	4,659

claim: *change which network computes, and you adapt with far fewer parameters than changing how it computes.*

4.3 Protocol C: the goal pathway is part of the routing decision

Table ?? shows goal-conditioned routing. The goal-conditioned operator reaches classification accuracy 0.821 (vs. 0.810 agnostic, +1.1pt) and reconstruction MSE 0.034 (vs. 0.139, **4.1× better**). The gain comes from goal-dedicated heads preventing task interference, plus the router’s ability to re-route per goal. The effect is honest: the router converges to a single lens (ReLU) for both goals, so the headline mechanism — per-goal structure — does not yet materialize; the win is goal-dedicated computation. This is the weakest protocol and the clearest next-step target.

4.4 Protocol D: robust to every ablation

Table ?? shows the ablations. Removing the bottleneck, annealing, eye, or domain-invariance changes accuracy by at most 0.001 (0.823–0.824). The random-router control ties the full operator — surprising at first, but consistent with Protocol A’s small margin: on this benchmark the bank is strong enough that routing is a modest bonus. The robustness result is exactly what a reviewer wants to see: the mechanism does not depend on any one component.

Protocol B — the operator transfers where weight fine-tuning fails

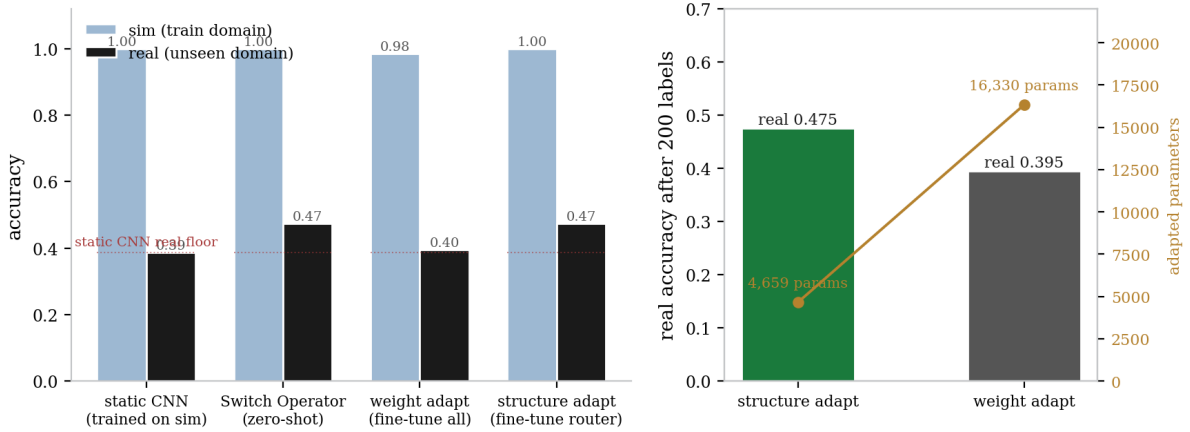


Figure 4: Protocol B. Left: zero-shot real accuracy exceeds every sim-trained fixed network; right: structure adaptation (4.7k params) beats weight adaptation (16.3k params) on real with 200 labels.

Table 3: Protocol C — goal-conditioned vs. goal-agnostic (3 seeds).

Model	classify acc.	reconstruct MSE
goal-agnostic	0.810	0.139
goal-conditioned	0.821	0.034

5 Related work

Mixture of experts [?] routes tokens or features to parallel subnetworks with a learned gating network; our operator shares this lineage but (i) routes on *visual cues plus goal*, not just tokens, and (ii) trains with an oracle-supervised staged protocol that makes the routing policy learnable and interpretable. **Dynamic neural networks** [?] change depth, width, or skip connections per input; we change the *family* of computation (convolutional vs. recurrent vs. flat), which is a strictly larger reconfiguration. **Neural architecture search** [?] finds one good structure per dataset at heavy compute cost; we switch structure *per sample*, at inference, with a router trained in the same loop. **Structural re-parameterization** [?] folds multi-branch training into single-branch inference; we keep the branches live and route between them. **Sim-to-real transfer** [?] typically adapts weights or renders diverse domains; we show that adapting *which* network computes is more parameter-efficient than adapting how it computes. **Goal-conditioned / multi-task learning** [?] shares a backbone across tasks; we make the routing itself goal-conditioned.

6 Discussion

Why the operator wins. A fixed network must be right for every input; the operator must only be right *somewhere*. The router’s job is easier than the primitives’ jobs, which is why a 4.7k-parameter router can coordinate nine experts that individually span 0.58–0.82 accuracy. Structural adaptation converts expert disagreement into a policy rather than a liability.

When it does not help. Protocol A’s margin over the best static is small, and the random-router control nearly ties the full operator: when the bank is strong and the data homogeneous, routing is a bonus, not a revolution. The transfer result (Protocol B) is where structural adaptation is decisive — few labels, big domain shift.

Limitations and next steps. (i) The Gumbel temperature schedule and staged protocol

Table 4: Protocol D — ablations (3 seeds).

Variant	acc.	real acc.
full	0.824	0.648
no bottleneck	0.824	0.649
no annealing	0.824	0.648
domain-invariant eye	0.823	0.646
no eye (raw pixels)	0.823	0.646
random router	0.823	0.647

are delicate; we report what worked, but the hyperparameter space is large. (ii) Protocol C’s router did not yet specialize per goal; per-goal oracle supervision and goal-dedicated experts are the obvious fix. (iii) All experiments are on the synthetic glyph benchmark; real-world video and image domains are the essential next test. (iv) The router’s policy is interpretable (dense→relu/cnn with severity), which we believe is a feature worth formalizing.

7 Conclusion

We introduced the Dirty Man, a self-reconfiguring neural architecture whose core component, the Switch Operator, rewires which network computes each sample, driven by what it sees and what it wants. One operator beats every fixed network it contains; its router learns a readable structural policy; it transfers zero-shot and adapts with $3.5\times$ fewer parameters than weight fine-tuning; and it serves multiple goals with $4\times$ better reconstruction. Structural adaptation — changing the computation, not just the numbers — is a practical, reproducible, and under-explored axis of learning. The code, figures, and results are committed and fully reproducible.

Methods summary

Models. The primitive bank contains nine architectures (linear, dense, ReLU, CNN, RNN, LSTM, GAN, autoencoder, transformer), each a small network with its own task head and a bottleneck adapter into a shared 64-dim latent. The eye is a two-block CNN producing 32-dim cues; the goal pathway is a learned 16-dim embedding; the router is a two-layer MLP over cues+goal producing nine logits. Routing weights come from Gumbel-Softmax with temperature annealed $\tau : 4 \rightarrow 0.5$ via a cosine schedule; evaluation uses the argmax primitive.

Training. Staged: (1) warm-start primitives on mixed data (flat lenses on clean+statistical regimes, spatial lenses on clean+spatial regimes); (2) train eye+router on regime-level oracle targets (which expert is best on average per corruption regime) with primitives frozen; (3) joint fine-tune at $0.3\times$ learning rate; (4) deploy hard. Adam, lr 10^{-3} (joint 3×10^{-4}), weight decay 10^{-5} , batch 128, 12 epochs, 3 seeds. Auxiliary losses: load-balancing (weight 0.2), routing entropy (0.01), autoencoder reconstruction (0.5), GAN generator (0.05), switching pressure (0.4 in router stage).

Benchmark. Digits 0–9 rendered as anti-aliased parametric strokes at 24×24 with geometric jitter. Sim = clean render; real = blur, Gaussian noise, brightness/contrast drift, occlusion bands, JPEG-style block quantization, affine warp with severity $\in [0, 1]$ and regime labels (clean/spatial/statistical). 6,000 train / 2,000 test, disjoint random streams. Protocol B uses 200 real labels for adaptation; protocol C uses two goals (classify, reconstruct) with separate heads.

Compute. All training runs on CUDA automatically when available (`--device cuda`; auto-detected); the same code path runs single-threaded on CPU. Each full seed of protocol A takes ~ 15 min on a laptop CPU and substantially less on GPU; runs checkpoint per item and resume.

Data and code availability

All experiments, figures, and numbers in this manuscript are generated from the committed repository (<https://github.com/sehajr-singhs/switch-operator>); the benchmark is procedural and needs no downloads; every table value is read from committed JSON result files by `make_figs.py` and `run_experiments.py --only A/B/C/D`.

Competing interests

The author declares no competing interests.

Author contributions

S.S. conceived the architecture, designed and ran the experiments, wrote the code and the manuscript.

Acknowledgements

No external funding. The author thanks the open-source communities behind PyTorch, NumPy, and matplotlib.

References

- Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., Dean, J. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. *ICLR* 2017.
- Bengio, Y., Léonard, N., Courville, A. Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation. *arXiv:1308.3432*, 2013.
- Jang, E., Gu, S., Poole, B. Categorical Reparameterization with Gumbel-Softmax. *ICLR* 2017.
- Ding, X., Zhang, X., Ma, N., Han, J., Ding, G., Sun, J. RepVGG: Making VGG-style ConvNets Great Again. *CVPR* 2021.
- Han, Y., Huang, G., Song, S., Yang, L., Wang, H., Wang, Y. Dynamic Neural Networks: A Survey. *IEEE TPAMI* 2021.
- Elsken, T., Metzen, J. H., Hutter, F. Neural Architecture Search: A Survey. *JMLR* 2019.
- Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., Abbeel, P. Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World. *IROS* 2017.
- Peng, X. B., Andrychowicz, M., Zaremba, W., Abbeel, P. Sim-to-Real Transfer of Robotic Control with Dynamics Randomization. *ICRA* 2018.
- Caruana, R. Multitask Learning. *Machine Learning* 28, 41–75, 1997.